AI Arena — Game SDK Developer Guide

> *This document is the authoritative reference for building games on the AI Arena platform. It covers the full game contract, SDK methods, bot and training interfaces, and publishing workflow. Game implementations — including the XO reference implementation — should be built and reviewed against this document alone.*

Overview

AI Arena is a multi-game platform. Games are independent packages that implement a strict contract. The platform loads any registered game via `React.lazy` and provides a runtime SDK object for all platform communication.

A game package has three responsibilities:

1. **Render the game** — a React component that receives `session` and `sdk` as props
2. **Describe itself** — a `meta` export with game metadata
3. **Support bots** — a `botInterface` export with AI and training logic (required if `meta.supportsBots` is true)

Games have no knowledge of sockets, auth, routing, or platform internals. All platform communication goes through the SDK.

Package Structure

```
packages/game-xo/
  package.json
  src/
    index.js          -- package entry point (exports default, meta, botInterface)
    GameComponent.jsx -- the playable React component
    meta.js           -- GameMeta export
    botInterface.js   -- BotInterface export
    logic.js          -- pure game logic (no platform dependencies)
    adapters.js       -- AI input/output adapters for packages/ai algorithms
```

package.json

```
{
  "name": "@aiarena/game-xo",
  "version": "1.0.0",
  "type": "module",
  "main": "./src/index.js",
  "exports": {
    ".": "./src/index.js"
  },
  "peerDependencies": {
    "react": "^19.0.0"
  }
}
```

Entry point (src/index.js)

```
export { default } from './GameComponent.jsx'
export { meta }         from './meta.js'
export { botInterface } from './botInterface.js'
```

The platform loads the package via:

```
React.lazy(() => import('@aiarena/game-xo'))
```

The Game Contract

Every game package must satisfy this shape:

```
{
  default:      React.ComponentType<{ session: GameSession; sdk: GameSDK }>
  meta:         GameMeta
  botInterface?: BotInterface   // required when meta.supportsBots is true
}
```

GameMeta

Static metadata the platform reads at registration time.

```
// src/meta.js
export const meta = {
  id:               'xo',
  title:            'Tic-Tac-Toe',
  description:      'Classic 3x3 strategy game. First to three in a row wins.',
  icon:             '/icons/xo.svg',
  minPlayers:       2,
  maxPlayers:       2,
  supportsBots:     true,
  supportsTraining: true,
  supportsPuzzles:  true,
  builtInBots:      [ /* BotPersona[] — see Bot Personas section */ ],
}
```

Field	Type	Notes
`id`	`string`	Stable, lowercase, unique across all games. Used as a key throughout the platform.
`title`	`string`	Human-readable name shown in UI.
`description`	`string`	One sentence shown on table cards and game detail views.
`icon`	`string?`	Path or URL to game icon.

`minPlayers`	`number`	Minimum seated players to start.
`maxPlayers`	`number`	Maximum players allowed at the table.
`supportsBots`	`boolean`	Enables bot opponent options at table creation.
`supportsTraining`	`boolean`	Enables Gym tab in the platform shell.
`supportsPuzzles`	`boolean`	Enables Puzzles tab in the platform shell.
`builtInBots`	`BotPersona[]`	Bot personalities bundled with the game. Empty array if `supportsBots` is false.

GameSession

The platform passes a `session` prop into every game component. It is read-only — never mutate it directly; use SDK methods instead.

```
interface GameSession {
  tableId:       string           // stable identifier for this table/match
  gameId:        string           // matches meta.id
  players:       Player[]         // all seated players
  currentUserId: string | null    // authenticated viewer; null for public
spectators
  isSpectator:   boolean          // true when currentUserId is not a seated player
  settings:      GameSettings     // table configuration set at creation
}

interface Player {
  id:          string
  displayName: string
  isBot:       boolean
}

type GameSettings = Record<string, unknown>  // game-defined keys
```

`isSpectator` is the key field for rendering mode. See Rendering Modes below.

GameSDK

The SDK object is passed as a prop alongside `session`. It is the only channel between a game and the platform.

```
interface GameSDK {
  submitMove(move: unknown): void
  onMove(handler: (event: MoveEvent) => void): () => void
  signalEnd(result: GameResult): void
  getPlayers(): Player[]
  getSettings(): GameSettings
  spectate(handler: (event: MoveEvent) => void): () => void
```

```
  getPreviewState(): unknown
  getPlayerState(playerId: string): unknown
}
```

sdk.submitMove(move)

Submit a move for the current player. The platform validates, commits, and broadcasts it.

- Throws if it is not the current player's turn
- Throws if the game has already ended
- `move` is game-defined — any JSON-serializable value

```
// XO example: move is the cell index 0-8
sdk.submitMove(cellIndex)
```

sdk.onMove(handler) --> unsubscribe

Register a handler that fires for every move (own or opponent). Returns an unsubscribe function for `useEffect` cleanup.

```
useEffect(() => {
  return sdk.onMove(event => {
    setBoard(event.state.board)
    setCurrentTurn(event.state.currentTurn)
  })
}, [sdk])
```

```
interface MoveEvent {
  playerId:  string    // who made the move
  move:      unknown   // the move value as passed to submitMove
  state:     unknown   // full game state after this move
  timestamp: string    // ISO timestamp from the server
}
```

sdk.signalEnd(result)

Signal that the game has ended. Call exactly once when win, draw, or forfeit is detected.

The platform records the result, updates ELO, and handles table cleanup.

```
sdk.signalEnd({
  rankings: [winnerId, loserId],  // ordered: winner first
  isDraw:   false,
})

// Draw:
sdk.signalEnd({ rankings: [], isDraw: true })
```

```
interface GameResult {
  rankings:  string[]                    // player IDs, winner first; empty for
draws
  isDraw:    boolean
  metadata?: Record<string, unknown>     // optional game-specific data (score, move
count, etc.)
}
```

sdk.getPlayers() --> Player[]

Returns the current seated players. Equivalent to `session.players` but reflects mid-game changes such as forfeits.

sdk.getSettings() --> GameSettings

Returns the table's configuration as set at creation. Values are game-defined.

```
// Table creation might set: { timeControlSeconds: 30 }
const { timeControlSeconds } = sdk.getSettings()
```

sdk.spectate(handler) --> unsubscribe

Identical to `onMove` but signals spectator intent to the platform. Use this instead of `onMove` when the viewer is not an active player.

```
useEffect(() => {
  if (session.isSpectator) {
    return sdk.spectate(event => setBoard(event.state.board))
  }
  return sdk.onMove(event => setBoard(event.state.board))
}, [sdk, session.isSpectator])
```

sdk.getPreviewState() --> unknown

Return a lightweight snapshot of the current board state. The platform calls this after each move to update the table card thumbnail on the Tables page.

- Must be cheap and synchronous
- Return only what is needed to render a preview — not the full game state
- Shape is game-defined and opaque to the platform

```
// XO example
sdk.getPreviewState = () => ({ board: [...board] })
```

sdk.getPlayerState(playerId) --> unknown

Return the portion of game state visible to the given player. Required for hidden-information games. For fully public games (XO, Connect4) return the full state.

The platform calls this before sending state to each client, ensuring players only see what they are entitled to see.

```
// XO: fully public — all players see the same state
sdk.getPlayerState = (playerId) => ({ board, currentTurn, winner })

// Poker (example): hole cards are private
sdk.getPlayerState = (playerId) => ({
  communityCards,
  pot,
  holeCards: playerId === ownerId ? myHoleCards : null,
})
```

Rendering Modes

The platform shell operates in two modes, derived automatically from `session.isSpectator`:

Mode	When	What the platform does
Focused	`isSpectator === false` (active player)	Full viewport, nav and sidebar hidden, floating "Back to Arena" escape affordance shown
Chrome-present	`isSpectator === true` (spectator, observer, replay viewer)	Nav and table context sidebar visible, game rendered in the content area

The game does not receive a mode prop. It reads `session.isSpectator` directly.

Required game behaviour by mode:

- **Focused:** all input enabled; game fills available space
- **Chrome-present:** all input disabled; game renders in a constrained content area; no cursor changes suggesting interactivity

```
// Pattern for handling both modes
function GameComponent({ session, sdk }) {
  const canPlay = !session.isSpectator

  return (
    <div className={session.isSpectator ? 'game-spectator' : 'game-focused'}>
      <Board
        onCellClick={canPlay ? handleCellClick : undefined}
        interactive={canPlay}
      />
    </div>
  )
}
```

Game Component

The default export is a standard React component. It receives exactly two props: `session` and `sdk`.

```jsx
// src/GameComponent.jsx
export default function GameComponent({ session, sdk }) {
  const [board, setBoard] = useState(Array(9).fill(null))
  const [currentTurn, setCurrentTurn] = useState('X')

  // Subscribe to moves
  useEffect(() => {
    return sdk.onMove(event => {
      setBoard(event.state.board)
      setCurrentTurn(event.state.currentTurn)
    })
  }, [sdk])

  function handleCellClick(index) {
    if (session.isSpectator) return
    sdk.submitMove(index)
  }

  return (
    <Board
      board={board}
      onCellClick={handleCellClick}
      interactive={!session.isSpectator}
    />
  )
}
```

Rules:

- No direct socket calls — all communication through `sdk`
- No platform imports (`auth` , `router` , `notificationStore` , etc.)
- No hardcoded player marks — derive from `session.players` and `session.currentUserId`
- `signalEnd` must be called exactly once when the game concludes

BotInterface

Required when `meta.supportsBots` is true. Exported as a named export from the package entry point.

```jsx
export const botInterface = {
  makeMove,
  getTrainingConfig,
  train,
  serializeState,
  deserializeMove,
  personas,
  GymComponent,
  puzzles,
}
```

Bot Personas

A persona is a named bot personality bundled with the game. The platform displays personas as opponent choices at table creation.

```
export const personas = [
  {
    id:          'minimax-easy',
    name:        'Easy Bot',
    description:'Makes mistakes on purpose. Good for beginners.',
    difficulty: 'easy',
    algorithm:  'minimax',
  },
  {
    id:          'minimax-hard',
    name:        'Perfect Play',
    description:'Never loses. Uses full-depth minimax.',
    difficulty: 'expert',
    algorithm:  'minimax',
  },
  {
    id:          'ql-trained',
    name:        'Trained AI',
    description:'Learns from experience. Improves with training.',
    difficulty: 'medium',
    algorithm:  'qlearning',
  },
]
```

```
interface BotPersona {
  id:          string    // stable identifier
  name:        string    // display name
  description: string    // short playstyle description
  difficulty:  'beginner' | 'easy' | 'medium' | 'hard' | 'expert'
  algorithm:   string    // 'minimax' | 'qlearning' | 'alphazero' | etc.
}
```

Dispatch guidance:

Current implementations may dispatch on `persona.id`. Future platform versions will support custom personas constructed at runtime — migrate dispatch to `persona.algorithm` + `persona.difficulty` when that work lands.

```
// Current: dispatch on id
function makeMove(state, playerId, persona, weights) {
  if (persona.id === 'minimax-easy') return minimaxBot(state, { depth: 2 })
  if (persona.id === 'minimax-hard') return minimaxBot(state, { depth: 9 })
  if (persona.id === 'ql-trained')   return qlBot(state, weights)
}
```

```
// Future-ready: dispatch on algorithm + difficulty
function makeMove(state, playerId, persona, weights) {
  if (persona.algorithm === 'minimax') {
    const depth = { easy: 2, medium: 5, hard: 7, expert: 9 }[persona.difficulty]
    return minimaxBot(state, { depth })
  }
  if (persona.algorithm === 'qlearning') return qlBot(state, weights)
}
```

botInterface.makeMove(state, playerId, persona, weights) --> move

Called server-side by the platform whenever a bot must act. Must be **synchronous** and **stateless**.

```
makeMove(
  state:    unknown,          // current game state
  playerId: string,           // the bot's player ID
  persona:  BotPersona,       // which persona is playing
  weights:  unknown | null,   // trained skill weights, or null if untrained
): unknown                    // a move value suitable for sdk.submitMove
```

```
function makeMove(state, playerId, persona, weights) {
  const mark = getMarkForPlayer(state, playerId)  // derive 'X' or 'O' from state

  if (persona.algorithm === 'minimax') {
    return minimaxMove(state.board, mark, persona.difficulty)
  }

  if (persona.algorithm === 'qlearning') {
    if (!weights) return randomMove(state.board)  // fallback if untrained
    const engine = new QLearningEngine()
    engine.loadQTable(weights)
    return engine.chooseAction(state.board, false)  // false = exploit only
  }
}
```

botInterface.getTrainingConfig() --> TrainingConfig

Called once when the Gym tab is opened. Returns the schema of algorithms and hyperparameters available for this game.

```
interface TrainingConfig {
  algorithm:       string                          // default algorithm
  defaultEpisodes: number
  hyperparameters: Record<string, HyperparameterDef>
}

interface HyperparameterDef {
  label:       string
  type:        'number' | 'select' | 'boolean'
  default:     unknown
```

```
  min?:        number      // for type: 'number'
  max?:        number
  step?:       number
  options?:    Array<{ value: string; label: string }>  // for type: 'select'
  description?: string
}
```

```
function getTrainingConfig() {
  return {
    algorithm:       'qlearning',
    defaultEpisodes: 5000,
    hyperparameters: {
      learningRate: {
        label:   'Learning Rate',
        type:    'number',
        default: 0.3,
        min:     0.01,
        max:     1.0,
        step:    0.01,
      },
      discountFactor: {
        label:   'Discount Factor',
        type:    'number',
        default: 0.9,
        min:     0.5,
        max:     1.0,
        step:    0.01,
      },
      decayMethod: {
        label:   'Epsilon Decay',
        type:    'select',
        default: 'exponential',
        options: [
          { value: 'exponential', label: 'Exponential' },
          { value: 'linear',      label: 'Linear' },
          { value: 'cosine',      label: 'Cosine' },
        ],
      },
    },
  }
}
```

The platform renders these as form controls in the Gym UI. The user adjusts values and hits Train. The platform then calls `train()` with a `TrainingRun` containing the resolved values.

botInterface.train(run, currentWeights, onProgress) --> Promise<TrainingResult>

Runs a training session server-side. May be long-running. Must call `onProgress` periodically so the platform can stream updates to the Gym UI.

```typescript
interface TrainingRun {
  algorithm: string
  episodes:  number
  params:    Record<string, unknown>  // resolved hyperparameter values
}

interface TrainingProgress {
  episode:        number
  totalEpisodes:  number
  outcome:        'WIN' | 'LOSS' | 'DRAW'
  epsilon?:       number
  avgQDelta?:     number
}

interface TrainingResult {
  episodesCompleted: number
  winRate:           number
  lossRate:          number
  drawRate:          number
  finalEpsilon?:     number
  weights:           unknown  // serialized weights to be stored in BotSkill.weights
}
```

```typescript
async function train(run, currentWeights, onProgress) {
  const engine = new QLearningEngine({
    learningRate:   run.params.learningRate,
    discountFactor: run.params.discountFactor,
    decayMethod:    run.params.decayMethod,
  })

  if (currentWeights) engine.loadQTable(currentWeights)

  let wins = 0, losses = 0, draws = 0

  for (let i = 1; i <= run.episodes; i++) {
    const result = runEpisode(engine, 'both', null)

    if (result.outcome === 'WIN')  wins++
    if (result.outcome === 'LOSS') losses++
    if (result.outcome === 'DRAW') draws++

    if (i % 100 === 0) {
      onProgress({
        episode:       i,
        totalEpisodes: run.episodes,
        outcome:       result.outcome,
        epsilon:       engine.epsilon,
        avgQDelta:     result.avgQDelta,
      })
    }
```

```
  }

  return {
    episodesCompleted: run.episodes,
    winRate:           wins   / run.episodes,
    lossRate:          losses / run.episodes,
    drawRate:          draws  / run.episodes,
    finalEpsilon:      engine.epsilon,
    weights:           engine.toJSON(),
  }
}
```

Resuming training: `currentWeights` is the previously stored value from `BotSkill.weights`. Pass `null` on first training. The platform handles persistence — the game just receives weights in and returns updated weights out.

botInterface.serializeState(state) --> unknown

Convert the current game state to a format suitable for storage and replay. Called by the platform on each move to build the move stream.

```
function serializeState(state) {
  return {
    board:       state.board,
    currentTurn: state.currentTurn,
    winner:      state.winner,
  }
}
```

botInterface.deserializeMove(raw) --> move

Convert a raw stored move back to the game's move representation. Used during replay and when re-hydrating state.

```
function deserializeMove(raw) {
  return Number(raw)  // XO moves are cell indices stored as numbers
}
```

GymComponent

The game's training UI. Rendered in the Gym tab of the platform shell. Required when `meta.supportsTraining` is true.

The platform passes a `GymProps` object:

```
interface GymProps {
  botId:               string           // bot being trained
  gameId:              string
  currentWeights:      unknown | null   // existing weights from BotSkill.weights
  onTrainingComplete:  (result: TrainingResult) => void
```

```
  onProgress?:         (progress: TrainingProgress) => void
}
```

The `GymComponent` is responsible for rendering training controls, triggering `botInterface.train()`, and calling `onTrainingComplete` when done. The platform persists the result.

```
export function GymComponent({ botId, gameId, currentWeights, onTrainingComplete,
onProgress }) {
  const [config, setConfig] = useState(null)

  useEffect(() => {
    setConfig(botInterface.getTrainingConfig())
  }, [])

  async function handleStartTraining(userParams) {
    const run = {
      algorithm: config.algorithm,
      episodes:  userParams.episodes,
      params:    userParams,
    }
    const result = await botInterface.train(run, currentWeights, onProgress)
    onTrainingComplete(result)
  }

  return <TrainingForm config={config} onStart={handleStartTraining} />
}
```

Puzzles

The game's curated puzzle set. Rendered in the Puzzles tab. Required when `meta.supportsPuzzles` is true.

```
interface Puzzle {
  id:           string
  title:        string
  description:  string
  difficulty:   'beginner' | 'intermediate' | 'advanced'
  initialState: unknown   // game-defined starting board position
  solution:     unknown   // correct move or array of equivalent moves
  playerMark:   string    // which player is to move ('X', 'O', etc.)
}
```

```
export const puzzles = [
  {
    id:           'xo-p1',
    title:        'Win in One',
    description:  'X can win immediately. Find the move.',
    difficulty:   'beginner',
    initialState: { board: ['X', 'O', 'X', null, 'O', null, null, null, null],
```

```
currentTurn: 'X' },
    solution:     6,   // cell index
    playerMark:   'X',
  },
  // ...
]
```

The platform renders puzzles using the game component with input restricted to a single move, then calls `deserializeMove` to compare the player's move against `solution`.

Using packages/ai

`packages/ai` is the platform-level shared AI library. Games import algorithms from it — they do not re-implement them.

Available algorithms:

Class	Algorithm	Best for
`QLearningEngine`	Q-Learning (tabular)	Small state spaces (XO, Connect4)
`SARSAEngine`	SARSA (tabular)	Same as Q-Learning
`DQNEngine`	Deep Q-Network	Larger state spaces
`NeuralNet`	Supervised neural net	Pattern recognition
`MonteCarloEngine`	Monte Carlo Tree Search	Strategy games
`PolicyGradientEngine`	Policy Gradient	Complex environments
`AlphaZeroEngine`	AlphaZero (MCTS + neural net)	Strongest play, highest compute
`minimaxImplementation`	Minimax + alpha-beta	Perfect play in small trees
`ruleBasedImplementation`	Heuristic rules	Fast, predictable difficulty

Import:

```
import { QLearningEngine, runEpisode, AlphaZeroEngine } from '@xo-arena/ai'
```

Game adapters (game-specific, implemented in `adapters.js`):

The algorithms are general-purpose. Games provide three adapter functions:

```
// serializeState: board --> model input (for neural net / DQN / AlphaZero)
function serializeState(board) {
  return board.map(cell => cell === 'X' ? 1 : cell === 'O' ? -1 : 0)
  // XO: 9-element vector. Connect4 would be 42-element, etc.
}

// deserializeMove: model output --> valid move index
function deserializeMove(output) {
```

```
  // output is a probability distribution over all actions
  // mask illegal moves (occupied cells) then take the argmax
  return legalMoves.reduce((best, idx) => output[idx] > output[best] ? idx : best)
}

// getLegalMoves: needed by MCTS, minimax, masked softmax
function getLegalMoves(board) {
  return board.map((cell, i) => cell === null ? i : -1).filter(i => i >= 0)
}
```

Multiple Skills Per Bot

Each bot can hold one skill per game:

- **XO bot** can have an XO skill (weights trained on XO)
- The same bot, when Connect4 ships, can earn a Connect4 skill
- Skills are independent — training XO does not affect Connect4

One skill per `(botId, gameId)` — enforced by a unique constraint. The skill records which algorithm was used (`BotSkill.algorithm`), so `makeMove` always knows how to deserialize the weights.

If a user wants the same game trained with a different algorithm, they create a new bot. Each bot has one identity, one playstyle, one algorithm per game.

Publishing to GitHub Packages

Game packages are published to the `@aiarena` scope on GitHub Packages.

`.npmrc` (in the game package root):

```
@aiarena:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=${GITHUB_TOKEN}
```

Publish:

```
npm publish
```

Platform consumes:

```
// bundled path (default — game ships with the platform)
React.lazy(() => import('@aiarena/game-xo'))

// split-out path (game deployed as a separate service)
React.lazy(() => import(/* @vite-ignore */
'https://games.aiarena.com/xo/bundle.js'))
```

Switching between paths is a one-line registry change. All games start bundled. A game is split out only when it warrants its own deployment.

XO Refactoring Checklist

XO is the reference implementation. Use this checklist to verify compliance:

GameContract exports

- [ ] `export default GameComponent` — React component, no platform imports
- [ ] `export { meta }` — all GameMeta fields populated including `builtInBots`
- [ ] `export { botInterface }` — full BotInterface implemented

GameComponent

- [ ] Receives only `{ session, sdk }` props
- [ ] No direct socket.io calls
- [ ] No auth imports
- [ ] Subscribes to moves via `sdk.onMove` or `sdk.spectate`
- [ ] Submits moves via `sdk.submitMove`
- [ ] Calls `sdk.signalEnd` exactly once at game end
- [ ] Respects `session.isSpectator` — disables input when true
- [ ] Derives player mark from `session.players` + `session.currentUserId` (no hardcoding)

GameSDK integration

- [ ] `sdk.getPreviewState()` returns a lightweight board snapshot
- [ ] `sdk.getPlayerState(playerId)` returns full state (XO is public information)
- [ ] `sdk.getSettings()` used for any table-level config (time control, etc.)

BotInterface

- [ ] `makeMove` is synchronous, stateless, accepts `(state, playerId, persona, weights)`
- [ ] `makeMove` dispatches on `persona.id` (plan to migrate to `persona.algorithm`)
- [ ] `makeMove` falls back to a default bot when `weights` is null
- [ ] `getTrainingConfig` returns config for all supported algorithms (Q-Learning, SARSA, DQN, Minimax, Rule-based, Monte Carlo, Neural Net, Policy Gradient, AlphaZero)
- [ ] `train` is async, calls `onProgress` every ~100 episodes, returns `TrainingResult` with `weights`
- [ ] `train` accepts `currentWeights` to resume from an existing skill
- [ ] `serializeState` and `deserializeMove` are implemented
- [ ] `personas` array is non-empty
- [ ] `GymComponent` renders training controls and calls `onTrainingComplete`
- [ ] `puzzles` array is populated

Packages

- [ ] No imports from `frontend/`, `backend/`, or `landing/` — standalone package only
- [ ] Game logic in `logic.js` is pure (no side effects, no platform dependencies)
- [ ] AI adapters in `adapters.js` implement `serializeState`, `deserializeMove`, `getLegalMoves`

Type Reference

All types are defined in `packages/sdk/src/index.d.ts`.

Type	Purpose
`GameContract`	Full shape a game package must export
`GameMeta`	Static game metadata
`GameSession`	Runtime session context passed to game component
`GameSDK`	Platform interface the game calls into
`Player`	A seated player (human or bot)
`GameResult`	Outcome reported via `signalEnd`
`MoveEvent`	A single move delivered to subscribers
`GameSettings`	Table configuration
`BotInterface`	Full bot and training contract
`BotPersona`	Named bot personality
`TrainingConfig`	Hyperparameter schema for Gym UI rendering
`TrainingRun`	Resolved training parameters passed into `train()`
`HyperparameterDef`	Single control descriptor for Gym UI
`TrainingProgress`	In-session progress update
`TrainingResult`	Final training output including weights
`GymProps`	Props passed into GymComponent by the platform
`Puzzle`	A single puzzle position